

EvoAPR: Enhancing Large Language Models for Automatic Program Repair with Genetic Algorithm and Dynamic LoRA

Huan Zhang
Central South University
Changsha, China
z_huan@csu.edu.cn

Qingyang Yan
Central South University
Changsha, China
8202210810@csu.edu.cn

Weihsuan Min
Central South University
Changsha, China
csu_semwh@csu.edu.cn

Chenyuan Zhang
Central South University
Changsha, China
8208220932@csu.edu.cn

Li Kuang*
Central South University
Changsha, China
kuangli@csu.edu.cn

Yingjie Xia
Hangzhou Dianzi University
Hangzhou, China
xiayingjie@hdu.edu.cn

Abstract—Automated Program Repair (APR) aims to automate the patch generation for buggy code and is vital in software development and maintenance. While large language models (LLMs) excel in various tasks, our empirical study shows they still face challenges in APR. LLMs take infilling templates with different qualities as input, and low-quality templates may misguide LLMs in constantly generating incorrect patches. Additionally, LLMs lack project-specific knowledge and struggle to leverage bug context fully. Therefore, we propose EvoAPR, which integrates the genetic algorithm and dynamic LoRA technique to enhance LLMs for better APR. First, to generate high-quality infilling templates, buggy codes are encoded to genetic representations, and genetic operators and multi-angle evaluation are designed to produce better templates. Then, to effectively utilize bug context, a bug-context-aware LoRA fine-tuning method is proposed to fuse various bug contexts by dynamically activating certain blocks of the LoRA module according to the bug index. Finally, these high-quality infilling templates are fed the fine-tuned LLMs for patch generation. Experimental results demonstrate that EvoAPR significantly improves LLMs' performance, surpassing state-of-the-art methods.

Index Terms—Automatic Program Repair, Project-level Patch Generation, Large Language Models, Genetic Algorithm, Dynamic LoRA.

I. INTRODUCTION

Software bugs are inevitable, and they not only limit software application and development but also potentially cause economic losses and threats to national and social security [1]. Due to the complexity of software systems, manual bug fixing is time-consuming and error-prone. Automated Program Repair (APR) aims to automatically generate patches for buggy code, thereby reducing time and labor costs.

Existing APR methods have evolved from traditional methods to learning-based methods and then to large language model (LLM)-based methods. Traditional methods [2], [3] often use predefined repair templates or rules for bug fixing,

but they are limited in the number and types of bugs they can address. Learning-based methods [4]–[11] typically apply a Neural Machine Translation architecture [12] to translate buggy code snippets into repaired versions. However, these methods exhibit weaker generalization when facing bug types not in the training data. LLM-based methods utilize LLMs pre-trained on billions of open-source code snippets to either directly generate complete code functions (generative-style APR) [13]–[16] or fill in correct code snippets within a given context (infilling-style APR) [17]–[19]. Currently, infilling-style APR methods have achieved state-of-the-art performance across multiple program repair datasets. Figure 1(a) shows an example of infilling-style APR. The buggy line is masked with `<SPAN>`, and the masked line and its surrounding context are input into the LLM. Then, the LLM can generate a patch by filling in the `<SPAN>` placeholder. We refer to the masked line as the *infilling template*.

Despite their advancements, LLMs still face challenges in APR: **1) Ignoring the quality of infilling templates.** LLMs take infilling templates as input to generate proper tokens. While various infilling templates can provide different infilling location information, their quality varies. Current methods ignore the differences in template quality, but the low-quality templates may misguide LLMs in constantly generating incorrect patches. **2) Unable to fully utilize bug context.** LLMs often lack project-specific knowledge, which hinders their ability to generate identifiers that have not been seen before in project-level program repair. While providing more extensive bug context (such as bug-related method information in a project) can help, LLMs still have difficulty effectively leveraging this context to produce proper patches due to input length limitations.

On this basis, we conduct an empirical study to explore the challenges mentioned above. Our study yields three key observations that can facilitate LLMs in the APR task. **First,**

*Corresponding author.

Bug-Fixing Code

```
public void process(Node externs, Node root) {
-   (new NodeTraversal(compiler, this)).traverse(root);
+   (new NodeTraversal(compiler, this)).traverseRoots(externs, root);
}
```

Buggy Line

Bug-Fixing Line

Cloze Input

```
public void process(Node externs, Node root) {
  (new NodeTraversal(compiler, this)).<SPAN>
}
```

Masked Span Token

Generated Patch

```
public void process(Node externs, Node root) {
  (new NodeTraversal(compiler, this)).traverseRoots(externs, root);
}
```

Generated Code

Fig. 1. LLM-based infilling-style APR.

the quality of infilling templates input into LLMs varies. Second, LLMs achieve their best performance when provided with high-quality templates. Third, many bug fixes require broader context information, including project-specific and bug-related methods and variables.

Based on these observations, we propose a novel APR approach, EvoAPR, which enhances LLMs for APR with the genetic algorithm and dynamic LoRA technique. Firstly, to generate high-quality infilling templates, we encode buggy code into genetic representations and design three genetic operators and a multi-angle evaluation to produce better templates. Next, to effectively utilize bug context, we construct a repair-oriented training dataset based on bug context, and we also improve the dynamic LoRA method to integrate various bug contexts by selectively activating specific blocks of the LoRA module depending on the bug index. Finally, these high-quality infilling templates are input into the fine-tuned LLMs for patch generation. In summary, the contributions of this paper are as follows:

- We conduct an empirical study and obtain three key observations that can facilitate LLMs in the APR task.
- We propose EvoAPR, which enhances LLMs for APR by designing the genetic algorithm to generate high-quality infilling templates and improving the dynamic LoRA technique to utilize bug context effectively.
- Experiment results demonstrate that EvoAPR significantly improves the performance of LLMs, surpassing state-of-the-art methods.

II. RELATED WORKS

Traditional APR methods include heuristic-based, constraint-based, and template-based approaches. Heuristic-based methods use manually defined rules, constraint-based methods rely on inferred program specifications, and template-based methods depend on predefined templates or strategies. Template-based methods have shown the most promise but are limited by their reliance on manually defined templates, restricting the number and types of bugs they can fix.

To address these limitations, learning-based APR methods have been proposed, utilizing NMT architectures and deep

learning models to learn transformations from buggy to repaired code. For instance, SequenceR [4] is an end-to-end program repair method based on sequence-to-sequence learning. CURE [5] uses an Neural Machine Translation (NMT) model that incorporates a programming language model, a code-aware search strategy, and a subword-tokenization technique to reduce the search space. DLFix [6] employs a tree-based RNN model to learn the context of repairs. CoCoNut [7] uses an ensemble of context-aware neural translation models. RewardRepair [8] introduces a loss function based on program compilation and testing information, making the generated patches more likely to compile and less prone to overfitting. Recoder [9] features a syntax-guided edit decoder with placeholder generation, implementing a novel provider/decider architecture to ensure syntactically correct patches. CIRCLE [10] is based on the T5 framework and supports continual learning across multiple programming languages. SelfAPR [11] changes the learning paradigm from supervised to self-supervised training. However, learning-based methods require supervised training datasets composed of buggy and repaired code snippets to learn the transformations effectively. Additionally, these methods may still not generalize to specific code or bug types not seen in bug-fixing datasets.

Recently, Large Language Models (LLMs) have shown remarkable general intelligence in various tasks. Building on this success, researchers have proposed LLM-based APR methods. These methods do not rely on repair datasets, and they leverage LLMs pre-trained on billions of open-source code snippets to either generate complete code functions (generative-style APR) directly or fill in correct code snippets within a given context (infilling-style APR). For instance, AlphaRepair [16] is the first infilling-style APR method to directly use large pre-trained code models for APR without fine-tuning/retraining on historical bug fixes. GAMMA [17] extracts templates based on Tbar [2], converts them into an infilling-style style, and uses UniXcoder to predict the correct code. Both Repilot [18] and FitRepair [19] incorporate project-specific information: Repilot utilizes a completion engine to obtain project information and proactively complete the code based on its recommendations, while FitRepair enhances the

model's understanding of project-specific information through knowledge-intensified fine-tuning, repair-oriented fine-tuning, and relevant identifier prompting strategies. However, these methods ignore the quality differences among infilling templates. Besides, they struggle to fully leverage bug context, especially in project-level program repair scenarios.

III. EMPIRICAL STUDY

In this section, we evaluate the impact of infilling template quality and bug context on program repair, aiming to address the following research questions:

- **RQ1:** What is the distribution of the quality of infilling templates?
- **RQ2:** Do high-quality infilling templates contribute to program repair?
- **RQ3:** Does a broader bug context contribute to program repair?

A. Dataset

We use the widely recognized Defects4J-v1.2 dataset [20] for evaluation, which includes 395 bugs from six open-source Java projects. We categorize these bugs into three types: single-line, single-hunk (consecutive lines), single-method bugs, representing fixes within a single line, hunk, method, respectively. Table I shows the distribution of these bug types.

TABLE I
DISTRIBUTION OF DIFFERENT BUG TYPES

Projects	Single-Line	Single-Hunk	Single-Method	Total
Chart	9	13	17	26
Closure	28	57	98	133
Lang	13	25	49	65
Math	24	36	82	106
Mockito	8	13	22	38
Time	3	7	20	27

From Table I, we can observe that single-line bugs account for 22% (85/395), while non-single-line bugs make up 78% (310/395). This distribution indicates that most bugs require more complex understanding and modification to fix. Thus, the dataset covers repair scenarios ranging from simple to complex, providing a foundation for comprehensive evaluation and comparison of APR methods.

B. RQ1: Distribution of Infilling Template Quality

Definition: To evaluate the quality of infilling templates, we define the low-quality and high-quality templates as follows.

- **Low-Quality Template:** is one where the masked span does not fully cover all bug-fixing tokens, so LLMs cannot produce proper tokens in the masked location for bug fixing.
- **High-Quality Template:** is one where the masked span fully covers all bug-fixing tokens.

For example, in Figure 1, the bug-fixing tokens are `traverseRoots` and `externs`, which appear after `(new NodeTraversal(compiler, this))`. There are two infilling templates: `(new`

`<SPAN>(compiler, this)).traverse(root);` and `(new NodeTraversal(compiler, this)).<SPAN>`. The first one is a low-quality template because the masked span (`<SPAN>`) is placed before `(new NodeTraversal(compiler, this))`. Thus, LLMs can not generate proper tokens in `<SPAN>` to produce the bug-fixing line. In contrast, the second one is a high-quality template because LLMs can generate `traverseRoots(externs, root);` to create the bug-fixing line effectively.

Setup: Referencing [19], we take each buggy hunk as input and employ three masking strategies (complete mask, partial mask, and template mask) to generate infilling templates. Since a bug in the dataset may have multiple buggy hunks, the dataset contains 1,069 buggy hunks. We categorize templates as follows: First, we generate the template set A (for buggy hunks) and the set B (for their corresponding repair hunks). Next, we define $A \cap B$ as the high-quality template set and $A - (A \cap B)$ as the low-quality template set.

Results: We count the number of buggy hunks whose low-quality templates account for the majority (at least 50%), as shown in Figure 2. We can find that 50% (534/1,069) buggy hunks whose low-quality templates account for at least 50%. This indicates that half of the buggy hunks are predominantly associated with low-quality templates.

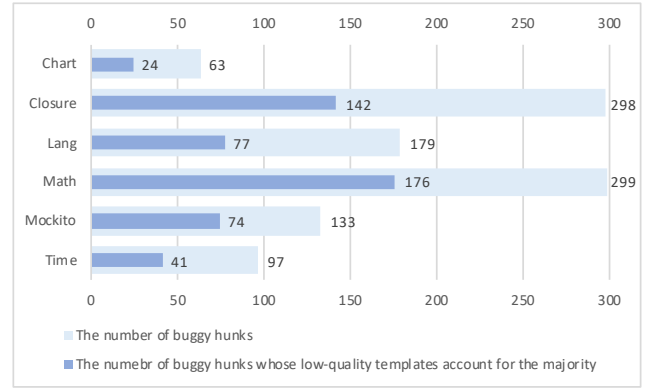


Fig. 2. Distribution of buggy hunks whose low-quality templates account for at least 50%.

Besides, considering some buggy hunks have only the simplest high-quality template `<SPAN>`, we count the number of these buggy hunks, as shown in Figure 3. We can observe that 63% (672/1,069) buggy hunks have only a high-quality template `<SPAN>`. This indicates that the type of high-quality templates is relatively simple.

Result-RQ1 The quality of infilling templates varies. Besides, high-quality templates are limited in type, and most buggy hunks have only a high-quality template `<SPAN>`.

C. RQ2: Effectiveness of High-Quality Templates

Setup: We evaluate three code-filling LLMs (CodeT5 [21], InCoder [22], and SantaCoder [23]) under three template

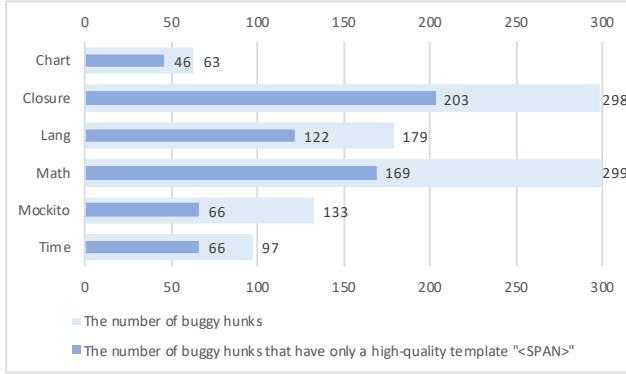


Fig. 3. Distribution of buggy hunks with only a high-quality template.

settings: 1) *Single Template* takes the high-quality template `<SPAN>` as input. 2) *Full Template* takes all low-quality and high-quality templates from RQ1 as input. 3) *Optimal Template* takes all high-quality templates from RQ1 as input. For a fair comparison, each LLM generates 100 patches for each bug under each template setting. We use the correct patch as the evaluation metric. The correct patch is semantically equivalent to the referenced developer patch.

TABLE II
PERFORMANCE OF LLMs FED IN TEMPLATES WITH DIFFERENT QUALITY

LLMs	Single Template	Full Template	Optimal Template
InCoder	35/31/38	43/52/46	53/55/55
SantaCoder	19/19/19	43/44/41	49/53/54
CodeT5	34/31/31	61/61/61	68/68/68

Results: Table II shows the results of three runs for each LLM under three template settings. We can find that all LLMs perform best under the optimal template setting, with InCoder, SantaCoder, and CodeT5 generating an average of 54, 52, and 68 correct patches, respectively. The performance trend is consistent across the three LLMs: the optimal template setting outperforms the full template setting, and the full template setting outperforms the single template setting. This pattern occurs because the full template setting provides more comprehensive and detailed information about repair locations than the single template setting, which only includes the basic high-quality template `<SPAN>`. In contrast, the optimal template setting, which excludes low-quality templates, allows LLMs to allocate more resources to high-quality templates, resulting in a greater number of correct patches.

Result-RQ2 Variety and high-quality infilling templates can enhance LLMs's program repair capability.

D. RQ3: Impact of Bug Context on Program Repair

Setup: We analyze the usage of different types of bug contexts in repaired code. Three types of bug contexts are considered in our evaluation: 1) *In-Method Context* includes

variable names defined within the buggy method. 2) *In-File Context* includes project-specific method names or global variable names within the buggy file. 3) *Cross-File Context* includes project-specific method names or global variable names from files imported by the buggy file. For each bug, we first extract the buggy files and their imported files from the buggy version. Then, we use Java Parser to analyze these files and identify in-method, in-file, and cross-file contexts. Lastly, we extract repaired hunks from the repaired version and identify the context type of the identifier used in bug fixes.

TABLE III
USAGE OF DIFFERENT BUG CONTEXT IN BUG FIXES

Projects	In-Method Context	In-File Context	Cross-File Context
Chart	20	3	0
Closure	59	11	31
Lang	46	2	0
Math	80	10	0
Mockito	20	2	4
Time	16	2	0
Total	241	30	35

Results: From Table III, we can find that 241 bug fixes rely only on in-method context, bug fixes rely on in-method and in-file contexts increases by 12%, and bug fixes rely on in-method, in-file, and cross-file contexts increases by 27%. The Closure project shows the most significant increase in different contexts: 59 bug fixes use in-method contexts, 70 bug fixes use in-method and in-file contexts (increasing by 19%), and 101 bug fixes use in-method, in-file, and cross-file contexts (increasing by 71%).

Result-RQ3 A broader bug context consisting of in-method, in-file, and cross-file information can benefit program repair.

IV. APPROACH

Based on the observations above, we propose EvoAPR, which integrates the genetic algorithm and dynamic LoRA technique for better APR. As shown in Figure 4, EvoAPR includes three key stages: 1) **Template Generation** - buggy hunks are extracted from buggy projects and encoded into genetic representations. These representations are then used to generate candidate infilling templates through our designed three genetic operations. Subsequently, a multi-angle evaluation is proposed to obtain high-quality templates from the candidates. 2) **Fine-tuning** - for each bug, extract bug context. These bug contexts are then used to create a repair-oriented training dataset and an index dataset mapping bugs to blocks of LoRA modules. The dynamic LoRA is improved to fuse various bug contexts by selectively activating specific LoRA blocks according to the bug index, enabling bug-related information to be stored as parameters in the corresponding LoRA blocks. 3) **Inference** - the high-quality infilling templates obtained from the template generation are used as the basis for inputting both the base LLM and the fine-tuned LLM obtained from the fine-tuning to generate corresponding patches.

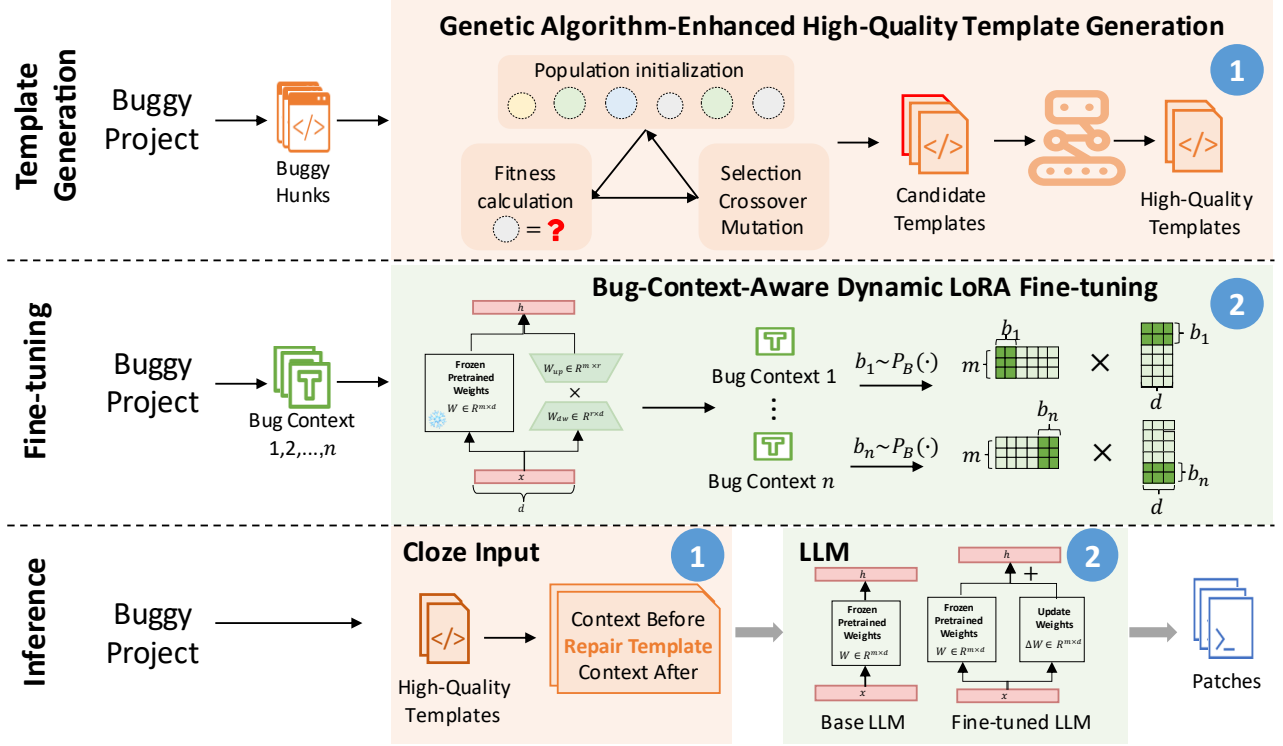


Fig. 4. EvoAPR overview.

A. High-Quality Template Generation

In the template generation phase, we employ a genetic algorithm to produce more effective infilling templates. Existing methods primarily rely on predefined masking strategies, typically masking only a single span (consecutive tokens) within a buggy hunk, making it difficult to accurately cover multiple repair locations. The genetic algorithm excels at searching for optimal solutions in complex combinatorial spaces, making it suitable for scenarios involving multiple repair locations. Through iterative optimization, it simultaneously addresses multiple repair points and efficiently explores the solution space, yielding more effective infilling templates.

We propose a genetic algorithm-enhanced high-quality template generation method, comprising three key components: population initialization, genetic operations, and template evaluation. First, buggy hunks are encoded into genetic representations to create an initial population. Next, selection, crossover, and mutation operations are used to enhance population diversity and quality. Finally, effective infilling templates are selected through multi-angle evaluation.

1) *Population Initialization*: Referencing ARJA [24], we encode buggy hunks as genetic representations in the form of triples (b, u, v) , where b , u , and v represent likely repair locations, operation types and repair ingredients, respectively. Each part of the triple is a vector of n integers, and n is the number of modification points. To identify these modification

points, we use Java Parser to parse a buggy hunk into multiple tokens and employ a bug locator, such as GPT-4, to identify the likely buggy tokens and assess their bug-proneness. The location of each likely buggy token can be seen as a modification point. Based on this, we perform the following initialization.

For the initialization of $b = (b_1, b_2, \dots, b_n)$, let bp_i represent the bug-proneness of the likely buggy token at the i th modification point. Then, b_i is initialized to 1 with probability $bp_i \times \mu$ and to 0 with probability $1 - bp_i \times \mu$, where $\mu \in (0, 1)$ is a predefined parameter. For the initialization of $u = (u_1, u_2, \dots, u_n)$, there are three kinds of operations: *delete*, *replace*, and *insert*. For each likely buggy token, we can either delete it, replace it with one token from the bug context, or insert one token from the bug context before it. u_i is randomly chosen from the set $[1, 2, 3]$, where 1, 2, and 3 refer to the delete, replace, and insert operations. For the initialization of $v = (v_1, v_2, \dots, v_n)$, v_i is randomly chosen from the set $[1, 2, \dots, p]$, where p is the number of tokens from the parsed bug context.

2) *Genetic Operators*: Based on the initial population, we apply selection, crossover, and mutation operators to generate new individuals, thereby enhancing both the diversity and quality of the population. We formulate the template generation as a multi-objective search problem. First, the fitness for each individual is calculated using the evaluation function in Sec.3.1.3. Then, the two individuals with the highest fitness

are selected as parents for crossover and mutation. To inherit desirable traits from the parents, crossover and mutation are applied separately to each part of the triple (individual). For the first part b , we use half uniform crossover and bit-flip mutation. For the remaining parts u and v , we employ single-point crossover and uniform mutation.

Using the original buggy hunks and genetic representations, we can transform each individual into infilling templates. Specifically, for modification points where b has a value of 0, the corresponding tokens are replaced with $\langle \text{SPAN} \rangle$. For modification points where b has a value of 1, the tokens are modified according to u and v . Non-modified points retain the original tokens from the buggy hunk, thereby generating the corresponding infilling templates.

3) *Template Evaluation*: We evaluate the quality of infilling templates from syntactic correctness and repair location coverage by combining expert knowledge and a bug locator. There are three key metrics: syntax rule compliance, buggy hunk masking rate, and repair location masking rate.

Syntax Rule Compliance (CRC): Based on syntax rules for Java program repair summarized by experts in the literature [17], [24], we design seven syntax rules to assess the syntactic correctness of infilling templates, as shown in Table IV. CRC measures the proportion of these rules satisfied by an infilling template. A higher CRC indicates better conformity to the expected syntax standards.

Bug Hunk Masking Rate (BHMR): An infilling template that masks more tokens in the original buggy hunk will likely cover the repair locations. For example, the infilling template " $\langle \text{SPAN} \rangle$ " that masks the entire buggy hunk can cover the repair locations and is a high-quality infilling template. BHMR measures the proportion of tokens masked by an infilling template in the buggy hunk. A higher BHMR indicates more comprehensive coverage. The calculation of BHMR is shown in Equation 1, where L represents the total number of tokens in the buggy hunk, and L_{um} denotes the number of unmasked tokens (non- $\langle \text{SPAN} \rangle$ tokens) in the infilling template.

$$BHMR = 1 - \frac{L_{um}}{L} \quad (1)$$

Repair Location Masking Rate (RLMR): Since BHMR cannot indicate whether the repair locations are precisely covered, we utilize a bug locator to identify potential repair locations and define RLMR to measure how the masked locations in an infilling template align with these identified locations. A higher RLMR indicates the infilling template has more coverage precision. The calculation of RLMR is shown in Equation 2, where $Mask$ is the tokens masked by the infilling template in the buggy hunk, $Locator(i)$ is the tokens related to i th repair location identified by the bug locator.

$$RLMR = \begin{cases} 0, & y = 0 \\ e^{1-y}, & y > 0 \end{cases} \quad (2)$$

$$y = \sum_{i=0}^n \frac{Mask \cap Locator(i)}{Locator(i)}$$

Based on the above metrics, we design a multi-angle evaluation function to score each infilling template obtained from the evolved population and those generated by the masking strategies of FitRepair and GAMMA. The calculation of this function is shown in Equation 3, where α_1 , α_2 , and α_3 are predefined parameters. Finally, the top-K infilling templates with the highest scores are selected as input for the LLMs.

$$score = \alpha_1 \times CRC + \alpha_2 \times BHMR + \alpha_3 \times RLMR \quad (3)$$

$$\alpha_1 + \alpha_2 + \alpha_3 = 1$$

B. Bug-Context-Aware Dynamic LoRA Fine-tuning

In the fine-tuning stage, we enhance dynamic LoRA to integrate diverse bug contexts, enabling LLMs to learn project-specific and bug-related knowledge while adapting to infilling-style APR tasks. Current strategies, such as prompting and fine-tuning, face limitations: prompting struggles with input length constraints, while fine-tuning requires extensive parameter optimization on large datasets.

LoRA [25], a lightweight training technique, reduces trainable parameters by summing pre-trained weights with low-rank adapter modules. Inspired by LoRA and DyLoRA, we construct training data from bug contexts rather than entire projects, storing bug-related information in the LoRA module. During inference, LLMs recall this knowledge to generate effective patches. Additionally, we build an index database to dynamically activate specific LoRA blocks based on bug contexts within the same project or type.

We propose a bug-context-aware dynamic LoRA fine-tuning method, comprising repair-oriented dataset construction and dynamic LoRA fine-tuning. The former creates a dataset from extracted bug contexts, while the latter trains specific LoRA blocks based on the bug index.

1) *Repair-Oriented Dataset Construction*: Infilling-style APR leverages the similarity between LLMs' pre-training tasks and the inference task to generate repair code. However, LLMs are not specifically trained for APR tasks. To address this, we construct a repair-oriented dataset to help LLMs adapt better.

When building the dataset, we extract bug-related context from the buggy project, including in-file context (non-buggy methods in the same file) and cross-file context (non-buggy methods from imported files). Incorporating both contexts enables LLMs to better understand the buggy code and generate more contextually relevant patches.

For each method in the extracted bug context, we randomly select a portion of the code lines and apply the mask strategies of FitRepair and GAMMA to generate infilling templates. For each generated infilling template RA , we concatenate the corresponding pre-context PRE and post-context PRO to form the input $X = PRE|RA|PRO$ for LLMs. The masked code span MS in the infilling template is the target output Y that LLMs need to predict. Based on this, a repair-oriented training dataset D_{train} is constructed, where each sample in D_{train} is represented as a tuple (X, Y) .

TABLE IV
SYNTAX RULES USED IN THE TEMPLATE EVALUATION

No.	Rule
1	Prefer modifying method and parameter names within “()” in method call statements.
2	Prefer modifying identifiers and operators within “()” in condition statements.
3	Do not replace a token with the same token.
4	Do not modify declared variables in variable declaration statements.
5	Do not insert tokens at the beginning of declaration statements.
6	Do not delete the return/throw keywords in return/throw statements.
7	Do not insert return/throw keywords at the beginning of non-return/throw statements.

2) *Dynamic LoRA Fine-tuning*: To effectively learn various bug contexts, as shown in Figure 5, unlike the original dynamic LoRA that randomly selects the range of LoRA ranks, we train non-overlapping LoRA modules for bugs in different projects while training certain blocks of the LoRA module for different bugs in the same project.

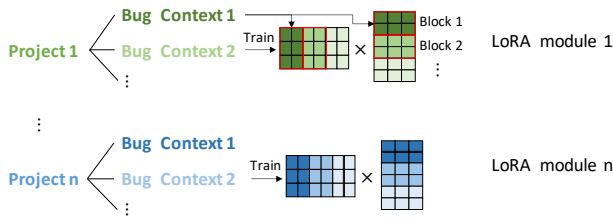


Fig. 5. Overall diagram of our dynamic LoRA.

Based on the constructed training dataset, we build an index dataset D_{index} mapping bugs and LoRA blocks. Each sample in D_{index} is represented as a tuple (p, id) , where p and id indicate the project name and the order of the bug in the project. Since different projects have varying coding styles and functions, we use non-overlapping LoRA modules for training across various projects. Besides, when repairing a specific bug, the focus is primarily on its context, while the contexts of other bugs within the same project can help LLMs adapt to the APR task during fine-tuning. Therefore, we use non-overlapping blocks of the same LoRA module for training across various bugs within the same project. Notably, the context information for all bugs in a project is stored in the corresponding LoRA module, and the context information for each bug is primarily stored in the corresponding block of the LoRA module.

For each LoRA module, assume have an up-projection $W_{up} \in \mathbb{R}^{m \times r}$ and a down-projection $W_{dw} \in \mathbb{R}^{r \times d}$, where $r \ll \min(m, d)$. For a bug with the index (p, id) , the bug-related training data is used to train the LoRA module W_{up}^p and W_{dw}^p related to the project p . Assume the rank of each block is r' , the LoRA block of the bug can be obtained by truncating W_{up}^p and W_{dw}^p , as shown in Equation 4.

$$\begin{aligned} W_{up\downarrow id}^p &= W_{up}^p[id \times r' : (id + 1) \times r', :] \\ W_{dw\downarrow id}^p &= W_{dw}^p[:, id \times r' : (id + 1) \times r'] \end{aligned} \quad (4)$$

, where $W_{up\downarrow id}^p$ and $W_{dw\downarrow id}^p$ are id -truncated versions of W_{up}^p and W_{dw}^p respectively. Then, the forward pass of the LoRA module can be calculated as Equation 5.

$$\begin{aligned} W_{up}^p &= \beta W_{up\downarrow id}^p + (1 - \beta) \text{avg} \left(\sum_{i=0, i \neq id}^n W_{up\downarrow i}^p \right) \\ W_{dw}^p &= \beta W_{dw\downarrow id}^p + (1 - \beta) \text{avg} \left(\sum_{i=0, i \neq id}^n W_{dw\downarrow i}^p \right) \\ h &= W_0 x + \frac{\alpha}{r} W_{up}^p W_{dw}^p x \end{aligned} \quad (5)$$

, where β is a hyper-parameter, α is a constant hyper-parameter for scaling, and W_0 is pre-trained weights matrix of the model. We set β greater than 0.5 to update the bug-related LoRA block more during training on specific bug data.

C. Inference

In the inference stage, the high-quality infilling templates obtained from the template generation stage are used as input for both the base LLMs and the fine-tuned LLMs obtained from the fine-tuning stage to generate patches. Based on previous research [19], we sample from each model in parallel to produce their respective patches, namely generating two potential patch lists for each bug using these two models. Notably, for the fine-tuned LLMs, we need to activate certain LoRA blocks according to the bug index, load the relevant LoRA module into the base LLMs as shown in Equation 5, and subsequently use the updated LLMs to generate patches.

To enhance patch validation, we rank the patches based on the outputs of each model. For each patch, we define an entropy to measure the extent to which the model will likely produce this patch. Let $T = \{t_1, t_2, \dots, t_n\}$ represent the list of tokens generated for a patch, and let $C(t_i)$ denote the probability of the model generating token t_i . The entropy is defined as Equation 6. We calculate this entropy for all patches and subsequently re-rank patches from the highest to lowest entropy. Each patch is then validated based on the ranking results.

$$\text{entropy}(T) = \frac{1}{n} \sum_{i=1}^n \log(C(t_i)) \quad (6)$$

V. EXPERIMENT

In this section, we study the following research questions to evaluate the effectiveness of our EvoAPR approach.

- **RQ4:** What is the performance of our EvoAPR approach?
- **RQ5:** Can template generation and fine-tuning stages improve EvoAPR’s performance?
- **RQ6:** How effective is our template generation method in generating high-quality templates?

A. RQ4 Overall Performance of EvoAPR

Setup: To validate the effectiveness of EvoAPR, we compare it with 13 classical and state-of-the-art APR methods, including a traditional APR method (TBar [2]), eight learning-based APR methods (SequenceR [4], CURE [5], DLFix [6], CoCoNuT [7], RewardRepair [8], Recoder [9], CIRCLE [10], and SelfAPR [11]), and four LLM-based APR methods (AlphaRepair [16], GAMMA [17], Repilot [18], and FitRepair [19]). For the ten baseline methods, we directly reference the results reported in their papers. For EvoAPR, we select CodeT5 as the base LLM because CodeT5 performs better than InCoder and SantaCoder in RQ2. For a fair comparison, we sample each model 5000 times and validate the top 2000 unique patches produced by each model (at most 4000 patches per bug). The model is trained based on the following hyper-parameters: the batch size is 4, the learning rate is $5e-5$, the epoch is 2, r' is 2, α is 2, and β is 0.9.

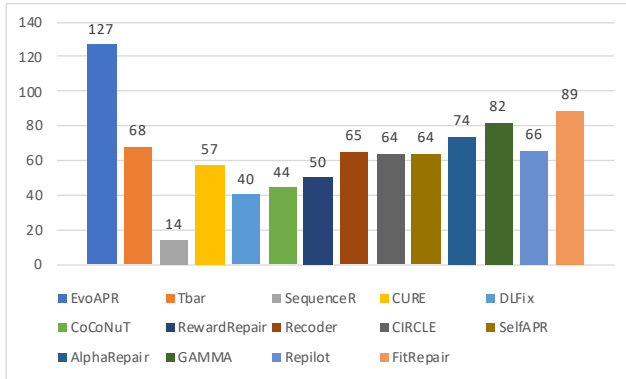


Fig. 6. Performance of Different APR Methods.

Result: Figure 6 presents the correct fixes of different APR methods. From Figure 6, we can find that EvoAPR achieves 127 bug fixes, surpassing the current state-of-the-art APR method by 13 additional fixes. Specifically, compared to the template-based method TBar, EvoAPR increases the number of bug fixes from 68 to 127, representing an 87% improvement. This increase occurs because TBar’s program repair capability is limited by manually defined templates. In contrast, EvoAPR utilizes the genetic algorithm to generate a variety of high-quality infilling templates to fix more bugs. Compared to the state-of-the-art LLM-based method FitRepair, EvoAPR raises the number of bug fixes from 89 to 127, representing a 43% improvement. This increase is attributed to the limited type and quality of infilling templates produced by FitRepair, and low-quality infilling templates can misguide LLMs in generating incorrect patches constantly. EvoAPR not only focuses on diverse and high-quality infilling templates but

also implements an effective dynamic LoRA to fuse various bug contexts, enhancing LLM’s performance by prompting and fine-tuning strategies.

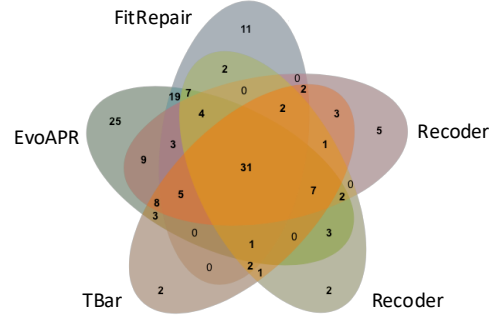


Fig. 7. Correct fix Venn diagram with different APR methods.

Figure 7 shows the number of unique bug fixes (that only one APR method can fix while others cannot) generated by EvoAPR compared with four top-performing APR baselines. From Figure 7, we can find that EvoAPR provides 25 additional bug fixes that the other four baselines cannot address. FitRepair, GAMMA, Recoder, and TBar only provide 11, 5, 2, and 2 additional bug fixes, respectively. This superior performance comes from the high-quality infilling templates generated by EvoAPR, which offer richer information about repair locations, effectively guiding the LLM in producing proper tokens. Besides, the bug-related context information from different bug types and projects can also help the LLM fix more bugs.

Result-RQ4 EvoAPR achieves 127 correct fixes on Defects4J-v1.2, surpassing the state-of-the-art APR method by more than 38 fixes.

B. RQ5 Ablation Experiment of EvoAPR

Setup: We conduct an ablation experiment on EvoAPR to evaluate the impact of the template generation and fine-tuning stages. The comparison variants include EvoAPR-tg, EvoAPR-ft, and EvoAPR-tg-ft. EvoAPR-tg excludes the template generation stage, EvoAPR-ft excludes the fine-tuning stage, and EvoAPR-tg-ft excludes both the template generation and fine-tuning stages, which is equivalent to using the base CodeT5.

Result: Table V presents the results of the ablation experiment. We can observe that EvoAPR achieves 127 bug fixes, indicating its superior performance compared to the other variants. Compared with EvoAPR-tg, bug fixes of EvoAPR increases by 35%. This indicates that excluding the template generation stage will significantly reduce the program repair capability of the LLM. Since EvoAPR-tg inputs many low-quality repair templates to the LLM, the LLM is hard to generate the correct patches based on these templates. While EvoAPR inputs various high-quality repair templates generated by the genetic algorithm to the LLM. These templates

TABLE V
THE CONTRIBUTION OF DIFFERENT STAGES IN EvoAPR

Projects	EvoAPR	EvoAPR-tg	EvoAPR-ft	EvoAPR-tg-ft
Chart	13	9	13	9
Closure	47	31	42	32
Lang	24	21	23	19
Math	35	26	34	25
Mockito	6	5	4	4
Time	2	2	2	2
Total	127	94	118	91

contain different and accurate repair location information, enabling the LLM to generate the correct patches more easily.

Compared with EvoAPR-ft, bug fixes of EvoAPR increases by 8%. It indicates that excluding the fine-tuning stage will reduce the program repair capability of the LLM. This is because EvoAPR trains the LLM on the repair-oriented dataset by the enhanced dynamic LoRA, which enables the LLM to adapt to the in-filling style APR task and effectively fuse various bug contexts. Removing the fine-tuning stage (EvoAPR-ft) yields 118 correct bug fixes. This result highlights that while the fine-tuning stage is effective, its impact is less significant than that of the template generation stage. Compared with EvoAPR-tg-ft, bug fixes of EvoAPR increases by 40%. We can observe that the performance drop becomes more pronounced when the repair generation and fine-tuning stages are removed. It is proved that both stages play crucial roles in the overall effectiveness of EvoAPR.

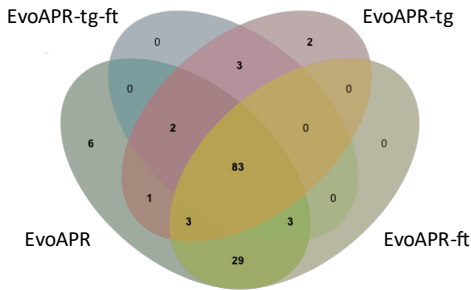


Fig. 8. Correct fix Venn diagram with different EvoAPR variants.

Figure 8 shows the number of unique bug fixes generated by EvoAPR compared with three EvoAPR variants. From Figure 8, we can find that EvoAPR provides 8 additional bug fixes that the three variants cannot address. Neither EvoAPR-tg-ft nor EvoAPR-ft produces any additional bug fixes. The limited performance of EvoAPR-tg-ft results from the weaker program repair capability of the base LLM compared to the evolved LLM. All bugs fixed by EvoAPR-ft can also be addressed by EvoAPR because EvoAPR integrates the results of the EvoAPR-ft and the fine-tuned LLM.

Result-RQ5 Both the template generation and fine-tuning stages can enhance the program repair capability of LLMs. Among these, the template generation stage has a particularly significant impact on the performance.

C. RQ6 Effectiveness of Our Template Generation Method

Setup: To investigate whether our proposed template generation method can effectively generate high-quality templates, we compare our method with two methods: 1) *Random Method* randomly selects K infilling templates from all infilling templates generated by the mask strategies of FitRepair and GAMMA; 2) *GPT Method* utilizes GPT-4’s instruction-following capability and general knowledge to select K infilling templates that are most likely to be high-quality from the infilling templates generated by the mask strategies of FitRepair and GAMMA. The instruction fed into GPT-4 is as follows:

Assuming you are an experienced software maintenance professional. Please answer my question with no explanation.

For the buggy code snippet after ###, firstly identify the reason for the bug in the code line marked with <BUG> and </BUG> based on the given code context; then provide three different fixed codes for the buggy code line; finally, from the templates list after @@@, select ten templates, which can be replaced only the with some tokens or blanks to generate the fixed code, and output ten the chosen template numbers in the form of [number, number, ...].

buggy code snippet

@@@ infilling templates

We set K to 10. When the number of infilling templates in the candidates is less than 10, we select all infilling templates present in the candidates. We use Acc@10 and Pre@10 as evaluation metrics. The calculation for Acc@10 is as follows: for each buggy hunk, if there is at least one high-quality infilling template, the Accuracy for that buggy hunk is 1; otherwise, it is 0. Acc@10 represents the average Accuracy across all bug hunks for each project. The calculation for Pre@10 is as follows: Precision indicates the proportion of high-quality infilling templates for a buggy hunk, and Pre@10 represents the average Precision across all bug hunks for each project.

Result: Table VI presents the Acc@10 and Pre@10 of different template generation methods across different projects. We can observe that our method outperforms both the random and GPT methods in terms of Acc@10 and Pre@10. Specifically, the Acc@10 in four projects (Chart, Closure, Mockito, and Time) is 1. It is indicated that our method can generate at least one high-quality infilling template in these projects. However, the Acc@10 in the Lang and Math projects is not 1. The values exceed that of the Random and GPT methods. Besides, for Pre@10, our method performs better

TABLE VI
PERFORMANCE OF DIFFERENT TEMPLATE GENERATION METHODS

Projects	Random Method		GPT Method		Our Method	
	Acc@10	Pre@10	Acc@10	Pre@10	Acc@10	Pre@10
Chart	0.87	0.68	0.94	0.69	1.0	0.76
Closure	0.89	0.63	0.88	0.64	1.0	0.68
Lang	0.87	0.66	0.91	0.67	0.99	0.73
Math	0.86	0.53	0.87	0.53	0.99	0.61
Mockito	0.92	0.58	0.94	0.59	1.0	0.70
Time	0.95	0.68	0.92	0.67	1.0	0.72
Total	0.89	0.61	0.90	0.61	1.0	0.68

than the other methods. For example, the Pre@10 of our method increases from 0.6753 to 0.7578 in the Chart project and increases from 0.5272 to 0.6050 in the Math project.

Overall, we can find that our method is superior to the Random and GPT methods. On the one hand, the infilling templates generated by the two methods are based on predefined templates, so the number and type of high-quality infilling templates are limited. On the other hand, they struggle to filter out high-quality infilling templates because the Random method randomly selects infilling templates, and the results of GPT are random. In contrast, our method can effectively filter out high-quality infilling templates by the multi-angle evaluation, which combines expert knowledge and a bug locator to evaluate each infilling template on syntactic correctness and repair location coverage. Therefore, our template generation method is superior to the random and GPT methods.

Result-RQ6 Compared with the random and GPT methods, our template generation method can generate more high-quality infilling templates across all projects.

VI. CONCLUSION

In this paper, we explore the limitations of LLMs in APR and obtain three key observations to facilitate LLMs. Based on these observations, we propose an EvoAPR approach to improve LLMs' program repair capabilities. The genetic algorithm is designed to generate high-quality infilling templates by multi-objective genetic evolution. Besides, a bug-context-aware dynamic LoRA method is proposed to fuse various bug contexts by activating specific LoRA blocks according to the bug index. Experimental results demonstrate that EvoAPR significantly enhances LLMs' performance, surpassing state-of-the-art methods.

ACKNOWLEDGMENT

This work has been supported by the National Natural Science Foundation of China under grant No.62472447, Hunan Provincial Natural Science Foundation of China under grant No.2024JK2006, Science and Technology Innovation Program of Hunan Province under grant No.2023RC1023, and the Fundamental Research Funds for the Central Universities of Central South University. Besides, this work was carried out in part using computing resources at the High Performance Computing Center of Central South University.

REFERENCES

- [1] J. Jiajun, C. Junjie, and X. Yingfei, "Survey of automatic program repair techniques," *Journal of Software*, vol. 32, no. 9, pp. 2665–2690, 2021.
- [2] K. Liu, A. Koyuncu, D. Kim, and T. F. Bissyandé, "Tbar: Revisiting template-based automated program repair," in *Proceedings of the 28th ACM SIGSOFT international symposium on software testing and analysis*, 2019, pp. 31–42.
- [3] A. Koyuncu, K. Liu, T. F. Bissyandé, D. Kim, J. Klein, M. Monperrus, and Y. Le Traon, "Fixminer: Mining relevant fix patterns for automated program repair," *Empirical Software Engineering*, vol. 25, pp. 1980–2024, 2020.
- [4] Z. Chen, S. Komrusch, M. Tufano, L.-N. Pouchet, D. Shoshvanyk, and M. Monperrus, "Sequencer: Sequence-to-sequence learning for end-to-end program repair," *IEEE Transactions on Software Engineering*, vol. 47, no. 9, pp. 1943–1959, 2019.
- [5] N. Jiang, T. Lutellier, and L. Tan, "Cure: Code-aware neural machine translation for automatic program repair," in *2021 IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*. IEEE, 2021, pp. 1161–1173.
- [6] Y. Li, S. Wang, and T. N. Nguyen, "Diflix: Context-based code transformation learning for automated program repair," in *Proceedings of the ACM/IEEE 42nd international conference on software engineering*, 2020, pp. 602–614.
- [7] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "Coconut: combining context-aware neural translation models using ensemble for program repair," in *Proceedings of the 29th ACM SIGSOFT international symposium on software testing and analysis*, 2020, pp. 101–114.
- [8] H. Ye, M. Martinez, and M. Monperrus, "Neural program repair with execution-based backpropagation," in *Proceedings of the 44th international conference on software engineering*, 2022, pp. 1506–1518.
- [9] Q. Zhu, Z. Sun, Y.-a. Xiao, W. Zhang, K. Yuan, Y. Xiong, and L. Zhang, "A syntax-guided edit decoder for neural program repair," in *Proceedings of the 29th ACM joint meeting on European software engineering conference and symposium on the foundations of software engineering*, 2021, pp. 341–353.
- [10] W. Yuan, Q. Zhang, T. He, C. Fang, N. Q. V. Hung, X. Hao, and H. Yin, "Circle: continual repair across programming languages," in *Proceedings of the 31st ACM SIGSOFT international symposium on software testing and analysis*, 2022, pp. 678–690.
- [11] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "Selfapr: Self-supervised program repair with test execution diagnostics," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.
- [12] I. Sutskever, "Sequence to sequence learning with neural networks," *arXiv preprint arXiv:1409.3215*, 2014.
- [13] S. D. Kolak, R. Martins, C. Le Goues, and V. J. Hellendoorn, "Patch generation with language models: Feasibility and scaling behavior," in *Deep Learning for Code Workshop*, 2022.
- [14] J. A. Prenner, H. Babii, and R. Robbes, "Can openai's codex fix bugs? an evaluation on quixbugs," in *Proceedings of the Third International Workshop on Automated Program Repair*, 2022, pp. 69–75.
- [15] C. S. Xia, Y. Wei, and L. Zhang, "Automated program repair in the era of large pre-trained language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 1482–1494.
- [16] C. S. Xia and L. Zhang, "Less training, more repairing please: revisiting automated program repair via zero-shot learning," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and*

Symposium on the Foundations of Software Engineering, 2022, pp. 959–971.

- [17] Q. Zhang, C. Fang, T. Zhang, B. Yu, W. Sun, and Z. Chen, “Gamma: Revisiting template-based automated program repair via mask prediction,” in *2023 38th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2023, pp. 535–547.
- [18] Y. Wei, C. S. Xia, and L. Zhang, “Copiloting the copilots: Fusing large language models with completion engines for automated program repair,” in *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2023, pp. 172–184.
- [19] C. S. Xia, Y. Ding, and L. Zhang, “Revisiting the plastic surgery hypothesis via large language models,” *arXiv preprint arXiv:2303.10494*, 2023.
- [20] R. Just, D. Jalali, and M. D. Ernst, “Defects4j: A database of existing faults to enable controlled testing studies for java programs,” in *Proceedings of the 2014 international symposium on software testing and analysis*, 2014, pp. 437–440.
- [21] Y. Wang, W. Wang, S. Joty, and S. C. Hoi, “Codet5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation,” *arXiv preprint arXiv:2109.00859*, 2021.
- [22] D. Fried, A. Aghajanyan, J. Lin, S. Wang, E. Wallace, F. Shi, R. Zhong, W.-t. Yih, L. Zettlemoyer, and M. Lewis, “Incoder: A generative model for code infilling and synthesis,” *arXiv preprint arXiv:2204.05999*, 2022.
- [23] L. B. Allal, R. Li, D. Kocetkov, C. Mou, C. Akiki, C. M. Ferrandis, N. Muennighoff, M. Mishra, A. Gu, M. Dey *et al.*, “Santacoder: don’t reach for the stars!” *arXiv preprint arXiv:2301.03988*, 2023.
- [24] Y. Yuan and W. Banzhaf, “Arja: Automated repair of java programs via multi-objective genetic programming,” *IEEE Transactions on software engineering*, vol. 46, no. 10, pp. 1040–1067, 2018.
- [25] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, “Lora: Low-rank adaptation of large language models,” *arXiv preprint arXiv:2106.09685*, 2021.